중앙대학교 소프트웨어학부 캡스톤디자인 2022

캡스톤디자인 멘토링 보고서 (2차)

팀 기본 정보

분반	조원 명단	멘토	멘토링 일시
7 분반	나종우, 장원범, 조준오	김지만	2022년 10월 17일
프로젝트 주제명	딥러닝 영상 분석을 활용한 클라이밍 경쟁 서비스		

교수님 피드백 결과 및 반영 계획

교수님 피드백 결과

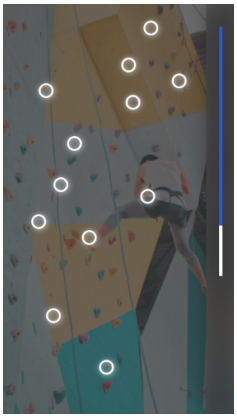
- ❍ Gripp 앱 요약 발표 진행
  - ❑ Gripp = Grip + People, 클라이밍 특화 소셜 미디어
  - ❑ 영상 분류와 편집 → 컴퓨터 비전으로 자동화
  - ❑ 지속적 동기 → 유저 간 경쟁 컨텐츠
- ❍ 컴퓨터 비전, 백엔드, 앱 파트별 진척 상황 보고
- ❍ 프로젝트 데모 관련 피드백
  - ❑ 최종 발표 시간은 10분, 그 중에 프로그램 데모 시간은 보통 5분 정도로 진행
  - ❑ 영상 업로드 시 주어진 시간 내 서버 처리 시간을 고려해야 함
- ❍ 컴퓨터 비전 관련 피드백
  - ❑ Object detection 성능 향상을 위해 더 많은 data augmentation 필요
- ❍ 프로젝트 협업 관련 피드백
  - ❑ 대체로 잘 진행되고 있는 것 같으나 파트별로 너무 별개로 진행하는 것 같음
  - ❑ 자기 역할 말고도 전체적인 프로젝트 구현 방법 이해 필요. 최종 발표는 한 명이 해야 됨

교수님 피드백 반영 계획

- ❍ 보통 프로그램 데모 시간이 5분 정도로 주어진 시간 내 서버 처리 시간을 고려해야 함
  - ❑ 처리 시간과 성능 사이에서 적절한 타협점을 찾는 중
  - ❑ 영상 게시 처리 100초 이내(업로드와 인코딩 50초, 컴퓨터 비전 50초)로 목표 설정
  - ❑ 데모 때 우선 영상부터 업로드하고, 다른 화면들 소개 후 인코딩이 끝나면 결과 보여주기
  - ❑ 모바일 네트워크 환경 상 파일 업로드 속도에는 한계가 있으니 작은 용량의 파일 업로드
- ❍ Object detection 성능 향상을 위해 더 많은 data augmentation 필요
  - ❑ Hold Detection에 적합하다고 판단한 horizontal flip, vertical flip, rotation 적용 예정
  - ❑ 추가 데이터셋 제작을 고려 중
- ❍ 프로젝트 협업 관련 피드백 반영
  - ❑ 팀 내에서 핵심 기능 위주로 구현 방법에 대해서 공유
  - ❑ 파트별로 각자의 구현 내용 발표보다는 각 기능 단위의 구현 내용 발표로 개선 예정

멘토링 결과 및 반영 계획

멘토링 결과
❍ 기존 SNS와 유사해 독창적이지 않은 앱의 디자인 ❑ 전반적으로 통일적이고 깔끔함 ❑ 다크 모드 디자인도 제작하여 사용자에게 배색 선택권을 주는 것이 보편적 ❑ 기존 SNS 사용법과 유사해, 처음 사용하는 사람들도 사용하기 편리할 것으로 예상 ❑ Pose Estimation 및 Detection 처리 과정을 표시하는 기능 제안 ❑ 기존 SNS에도 영상을 업로드하는 기능 제안 ❍ 클라이밍 영상에서 Pose estimation 성능 ❑ 손/발을 인식을 위해 pose estimation보다 object detection이 더 안정적일 것 ❑ 클라이밍 영상이라는 특수성으로 pose estimation의 고질적인 문제가 드러남 ❍ 낮은 hold detection의 성능 ❑ 데이터셋을 보강하는 것이 효과적 ❑ 쉬운 cases는 그대로 두고 어려운 cases에 해당하는 이미지들을 집중적으로 보강 ❑ Triplet loss를 적용할 수 있으나 데이터 보강이 더 효율적일 것이라 판단 ❑ Image resolution을 향상시키거나 bagging을 사용할 수 있으나 데이터셋 보강을 추천

멘토링 반영 계획
❍ 디자인 관련 피드백 반영 ❑ 현재 디자인에 다크 모드 등 테마를 적용하는 기능을 추가 예정 ❑ Pose Estimation 및 Detection 처리 과정 화면  - 멘토링 후 왼쪽 사진과 같은 UI를 연상 - 영상 인코딩이 비동기적으로 이뤄져서 로직 고민 필요. 추후 기능 추가 여부 결정 예정 ❑ 타 SNS에도 영상을 업로드하는 기능 - 기존 기능으로 서비스 안정화 후 기능 추가 여부 결정 예정 ❍ 클라이밍 영상에서 pose estimation 성능 관련 피드백 반영 ❑ Pose estimation이 아닌 다른 방법을 시도 예정 ❑ 손의 bounding box를 annotation한 데이터 탐색 및 학습 예정 ❍ 낮은 hold detection의 성능 관련 피드백 반영 ❑ 추가적인 data annotation을 목표로 잡음 ❑ Background만 존재하는 이미지 추가 예정

진행상황

진행상황
❍ 가입/로그인 ❑ 회원가입, 로그인, 토큰 관련 API 개발 완료 - 로그인 API로 refreshToken을 얻고, refreshToken으로 accessToken 갱신 가능 - accessToken으로 서비스 내 API 호출 가능 (인증, 인가 적용) ❑ SwiftUI 구성 중 ❍ 홈 화면 (피드) ❑ 게시물 검색 API 개발 중 - 제목, 유저, 난이도, 벽 각도, 날짜, 상태(인코딩 중, 등반 성공/실패) 등 다중 조건 검색 - 최신순, 시간순, 조회순, 좋아요 순, 어려운 순, 쉬운 순으로 정렬 가능 ❑ SwiftUI 구성 완료, UI 이벤트 처리 중, 서버 유즈 케이스와 베이스 작성 중

❍ 리더보드

❑ 리더보드 API 개발 완료 (유저의 점수, 티어 기반 랭킹)

- 점수는 유저의 등반 성공 영상 중 어려운 순으로 10개까지 나열하여 가중치 합 계산
- 상위 리더보드(전체 상위 10명)와 내 근처 리더보드(상위 10명부터 하위 10명까지) 지원

❑ SwiftUI 구성 완료, UI 이벤트 처리 중, 서버 유즈 케이스와 베이스 작성 중

❍ 내 정보

❑ 게시물 검색 API 개발 중

- 홈 화면에서 쓰는 게시물 검색 API 재활용
- 검색 API 호출 시 유저 조건으로 검색하면 내 정보 화면 구현 가능

❑ 홈 화면과 유사하지만 내 게시글과 기타 필요 정보만 보이도록 개발. SwiftUI 구성 완료, UI 이벤트 처리 중, 서버 유즈 케이스와 베이스 작성 중

❍ 게시물 등록

❑ 영상 업로드 과정 서버 측 플로우 개발 완료 (점진적으로 성능 개선 예정)

- 앱에서 웹 서버로 영상 업로드 → 웹 서버에서 비디오 서버로 영상 전송 → 비디오 서버에서 딥러닝 서버로 예측 요청 → 비디오 서버에서 예측 결과를 활용해 인코딩 → 클라우드에 스트리밍용 파일 및 썸네일 업로드 → 웹 서버로 결과 전송
- 영상 처리는 오래 걸리는 작업이므로 비동기 방식을 사용함. 웹 서버와 비디오 서버 사이 통신을 Message Queue(RabbitMQ)로 진행하여 비동기 통신을 구현함. 따라서 웹 서버에 영상 업로드를 했을 때, 인코딩 대기 시간 없이 일단 즉시 앱으로 응답이 보내짐. 앱에서는 영상 인코딩이 될 때까지 로딩을 기다리지 않아도 됨. 영상 업로드 후 인코딩이 되기 전에는 피드에서 영상이 인코딩 중으로 나오다가 인코딩이 완료되면 볼 수 있는 방식
- 딥러닝은 YOLO 계열 모델 학습 및 비교 후 base model을 yolov5n으로 결정함. 이를 토대로 Detection module, Pose estimation module을 개발 완료함. 개발한 모듈로 (1)등반 성공 여부 분류, (2)등반 시작과 끝 시간 추출을 제공하는 딥러닝 API를 개발 완료
- 딥러닝 성능 향상을 위한 knowledge distillation 구현 완료
- 딥러닝 성능과 소요시간을 고려해 동영상에서 0.5초 마다 frame을 추출하여 처리
- 영상 인코딩과 썸네일 추출은 FFmpeg로 구현. 앱에 제공하는 영상은 느린 다운로드 방식이 아니라 빠른 시청이 가능한 스트리밍 방식으로 제공됨. 영상을 n초 단위로 분할하여 스트리밍 가능한 HLS 포맷으로 인코딩함. HLS은 720p와 480p 화질로 각각 인코딩하여 그때그때 모바일 네트워크 환경에 맞게 화질이 변화하는 Adaptive Streaming 제공

❑ SwiftUI 구성 완료, UI 이벤트 처리 중, 서버 유즈 케이스와 베이스 작성 중

❍ 게시물 조회

❑ SwiftUI 구성 완료, UI 이벤트 처리 중, 서버 유즈 케이스와 베이스 작성 중

❍ 기타

❑ 비디오 서버, 딥러닝 서버 배포 시작

❑ 비디오 서버는 NestJS(Node.js 프레임워크)로 구현

❑ 비디오 서버는 웹 서버와 함께 Docker로 배포. Docker Compose로 다중 컨테이너 관리

❑ 딥러닝 서버는 Oracle Cloud에서 배포